

Федеральное государственное автономное образовательное учреждение
высшего образования

«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Лабораторная работа № 3

Базы данных

Выполнил

Добкес Михаил Константинович,

группа Р3106

Проверил

Преподаватель Мартин Райла

Санкт-Петербург, 2026

Содержание

Выполнение работы.....	3
1. Построение исходной схемы отношений.....	3
Предметная область.....	3
2. Построение неудачного решения.....	6
3. Определение функциональных зависимостей.....	9
4. Приведение к 3NF.....	20
5. Описание изменений после приведения к 3NF.....	22
6. Проверка на BCNF.....	23
7. Сравнение схем, денормализация.....	27
8. Триггер.....	30
Пример 3NF, но не BCNF.....	36
Заключение.....	38

Выполнение работы

1. Построение исходной схемы отношений

Предметная область

Описываются события периода подавления восстания Джеймса Скотта, герцога Мортмута против короля Якова II в 1685 году. Включены репрессии после Битвы при Седжмуре, массовые казни, суды и казни. Центральным объектом является обвиняемый в государственной измене врач Питер Блад.

Перечисление отношений

human - хранит информацию о людях, участвующих в событиях, включая их состояние и профессию.

атрибуты: id, is_sane, courage, job, first, middle, last

ключи: PK id

prison - хранит информацию о тюрьмах, их условиях, расположении

атрибуты: id, conditions, location_id

ключи: PK id, FK location_id → location(id)

event - хранит информацию о событиях и их результатах

атрибуты: id, is_successfull, location_id

ключи: PK id, FK location_id → location(id)

court - хранит информацию о судах, рассматривающих дела обвиняемых

атрибуты: id, name, location_id, location_name

ключи: PK id, FK location_id → location(id)

factor - хранит факторы, влияющие на обвинения и наказания, а также время конца/начала

атрибуты: id, type, start_time, end_time

ключи: PK id

accusation - хранит сведения об обвинениях, включая тип и суд

атрибуты: id, human_id, type, accused_time, court_id, court_name

ключи: PK id, FK human_id → human(id), FK court_id → court(id)

punishment - хранит сведения о наказаниях, включая тип и суд

атрибуты: id, accusation_id, type, execution_time

ключи: PK id, FK accusation_id → accusation(id)

wound - хранит данные о ранениях людей и их исходе

атрибуты: id, human_id, is_fatal, dt, is_cured

ключи: PK id, FK human_id → human(id)

location - хранит информацию о местах, где происходят события, а также расположены тюрьмы

атрибуты: id, name, climate

ключи: PK id

event_people - связывает людей с событиями и фиксирует их роль и последствия участия.

атрибуты: event_id, human_id, role, consequence

ключи: PK (event_id, human_id), FK event_id → event(id), FK human_id → human(id)

relationships - содержит информацию об отношениях между людьми

атрибуты: human1_id, human2_id, type

ключи: PK (human1_id, human2_id), FK human1_id → human(id), FK human2_id → human(id)

factor_accusation - связывает факторы с конкретными обвинениями

атрибуты: factor_id, accusation_id

ключи: ПК (factor_id, accusation_id), FK factor_id → factor(id), FK
accusation_id → accusation(id)

factor_punishment - связывает факторы с конкретными наказаниями

атрибуты: factor_id, punishment_id

ключи: ПК (factor_id, punishment_id), FK factor_id → factor(id), FK
punishment_id → punishment(id)

court_judge - связывает суды с судьями

атрибуты: court_id, human_id

ключи: ПК (court_id, human_id), FK court_id → court(id), FK human_id →
human(id)

prison_human - связывает людей с тюрьмами и фиксирует периоды их
заключения

атрибуты: human_id, prison_id, imprisoned_time, released_time

ключи: ПК (human_id, prison_id, imprisoned_time), FK human_id → human(id),
FK prison_id → prison(id)

2. Построение неудачного решения

Перечисление отношений

human

атрибуты: id, name, job, court_name, court_location

court_location: NOT NULL

ключи: PK id

case

атрибуты: case_id, human_id, human_name, accusation_type, court_name,
court_location

ключи: PK case_id

punishment

атрибуты: id, case_id, human_name, punishment_type, execution_time

ключи: PK id

event

атрибуты: id, human_id, human_name, event_result, event_location

ключи: PK id

Ошибки

1. human_name дублируется
2. court_name, court_location повторяются
3. одни и те же сущности(человек, суд) распределены по разным таблицам
4. пропущены некоторые FK
5. Данные не согласованы, а значит, могут возникнуть противоречия

Примеры

Аномалии вставки

court_location: NOT NULL

INSERT:

INSERT INTO human

(id, name, job, court_name)

VALUES (1, 'IVANOV IVAN', 'killer', 'court 1')

Будет ошибка вставки, так как требуется обязательный атрибут - court_name, который ошибочно был сделан NOT NULL

Аномалии модификации

UPDATE:

UPDATE human

SET name = 'Peter Blood'

WHERE id = 1;

Так как в case, event, punishment имя не изменится - данные расходятся

Другой пример:

UPDATE case

SET court_location = 'London'

```
WHERE case_id = 1;
```

```
UPDATE human
```

```
SET court_location = 'Bristol'
```

```
WHERE id = 1;
```

Данные снова расходятся

Аномалии удаления

```
DELETE:
```

```
DELETE FROM case
```

```
WHERE case_id = 1;
```

Может исчезнуть информация о суде, хотя удалили только 1 дело

3. Определение функциональных зависимостей

human

Поскольку люди могут иметь одинаковые атрибуты, ни один из атрибутов не гарантирует уникальность. Поэтому - используется искусственный ключ - id

При этом каждому человеку соответствует только 1 набор атрибутов

Кандидатный ключ: id

Функциональные зависимости:

id - first, middle, last, job, is_sane, courage

Минимальное покрытие:

id - first

id - middle

id - last

id - job

id - is_sane

id - courage

Частичных зависимостей нет, потому что ключ простой (id).

Транзитивных зависимостей нет, так как все атрибуты зависят напрямую от id.

Все неключевые атрибуты зависят только от ключа id.

Нарушений нормальных форм нет.

prison

Так как тюрьма является отдельной сущностью и может иметь уникальные условия содержания и местоположение, используется искусственный ключ id. Условия и расположение относятся к самой тюрьме и не могут отличаться в рамках одной записи.

Кандидатный ключ: id

Функциональные зависимости:

id - conditions, location_id

Минимальное покрытие:

id - conditions

id - location_id

Частичных зависимостей нет, ключ простой (id).

Транзитивных зависимостей нет, атрибуты не зависят друг от друга (conditions, location_id независимы).

Все атрибуты зависят только от id.

Нарушений нормальных форм нет.

location

Поскольку каждое место является отдельной сущностью и может иметь одинаковые названия или климатические условия с другими местами, используется искусственный ключ id. Название и климат уникальны для конкретного места.

Кандидатный ключ: id

Функциональные зависимости:

id - name, climate

Минимальное покрытие:

id - name

id - climate

Частичных зависимостей нет, ключ простой (id).

Транзитивных зависимостей нет, name и climate не зависят друг от друга.

Все неключевые атрибуты зависят от id.

Нарушений нормальных форм нет.

event

Поскольку каждое событие фиксируется отдельно и может происходить в одном и том же месте или иметь одинаковый результат, используется искусственный ключ id. Результат и место проведения относятся к самому событию.

Кандидатный ключ: id

Функциональные зависимости:

id - is_successfull, location_id

Минимальное покрытие:

id - is_successfull

id - location_id

Частичных зависимостей нет, ключ простой (id).

Транзитивных зависимостей нет, is_successfull и location_id не связаны функционально.

Все атрибуты зависят только от id.

Нарушений нормальных форм нет.

court

Поскольку каждый суд является отдельной сущностью и может иметь одинаковые названия с другими судами, используется искусственный ключ id. Название и местоположение относятся к самому суду. При этом название места определяется через location_id.

Кандидатный ключ: id

Функциональные зависимости:

id - name, location_id, location_name

location_id - location_name

Минимальное покрытие:

id - name

id - location_id

Нарушение нормальных форм(3NF):

id - location_id

location_id - location_name

id - location_name

Нарушение связано с наличием транзитивной зависимости, в результате чего происходит дублирование location_name.

factor

Поскольку каждый фактор является отдельной сущностью и может иметь одинаковый тип или временные границы с другими факторами, используется искусственный ключ id. Все атрибуты относятся к самому фактору.

Кандидатный ключ: id

Функциональные зависимости:

id - type, start_time, end_time

Минимальное покрытие:

id - type

id - start_time

id - end_time

Частичных зависимостей нет, ключ простой (id).

Транзитивных зависимостей нет, type, start_time, end_time независимы между собой.

Все атрибуты зависят от id.

Нарушений нормальных форм нет.

accusation

Поскольку каждое обвинение фиксируется как отдельная запись и может иметь одинаковые типы и времена с другими обвинениями, используется искусственный ключ id. Обвинение связано с конкретным человеком и судом. Название суда определяется через court_id.

Кандидатный ключ: id

Функциональные зависимости:

id - human_id, type, accused_time, court_id, court_name

court_id - court_name

Минимальное покрытие:

id - human_id

id - type

id - accused_time

id - court_id

court_id - court_name

Нарушение нормальных форм:

id - court_id

court_id - court_name

punishment

Поскольку каждое наказание является отдельной сущностью и может иметь одинаковые типы и времена исполнения, используется искусственный ключ id. Все атрибуты относятся к самому наказанию.

Кандидатный ключ: id

Функциональные зависимости:

id - accusation_id, type, execution_time

Минимальное покрытие:

id - accusation_id

id - type

id - execution_time

Частичных зависимостей нет, ключ простой (id).

Транзитивных зависимостей нет, type и execution_time не зависят друг от друга.

Все неключевые атрибуты зависят от id.

Нарушений нормальных форм нет.

wound

Поскольку каждое ранение фиксируется отдельно и может иметь одинаковые характеристики у разных людей, используется искусственный ключ id. Все атрибуты относятся к конкретному ранению.

Кандидатный ключ: id

Функциональные зависимости:

id - human_id, is_fatal, dt, is_cured

Минимальное покрытие:

id - human_id

id - is_fatal

id - dt

id - is_cured

Частичных зависимостей нет, ключ простой (id).

Транзитивных зависимостей нет, атрибуты (is_fatal, dt, is_cured) независимы.

Все атрибуты зависят от id.

Нарушений нормальных форм нет.

event_people

Поскольку участие человека в событии определяется одновременно событием и человеком, используется составной ключ. Роль и последствия зависят только от конкретного человека и конкретного события.

Кандидатный ключ: (event_id, human_id)

Функциональные зависимости:

(event_id, human_id) - role, consequence

Минимальное покрытие:

(event_id, human_id) - role

(event_id, human_id) - consequence

Частичных зависимостей нет, ключ составной и используется полностью.

Транзитивных зависимостей нет, role и consequence зависят только от пары (event_id, human_id).

Все неключевые атрибуты зависят от полного ключа.

Нарушений нормальных форм нет.

relationships

Поскольку тип отношения одного человека к другому определяется конкретной парой людей, используется составной ключ.

Кандидатный ключ: (human1_id, human2_id)

Функциональные зависимости:

(human1_id, human2_id) - type

Минимальное покрытие:

(human1_id, human2_id) - type

Частичных зависимостей нет, ключ составной и используется целиком.

Транзитивных зависимостей нет, type зависит только от пары (human1_id, human2_id).

Все атрибуты зависят от полного ключа.

Нарушений нормальных форм нет.

factor_accusation

Поскольку связь отражает отношение многие-ко-многим и не содержит дополнительных атрибутов, используется составной ключ.

Кандидатный ключ: (factor_id, accusation_id)

Функциональные зависимости:

(factor_id, accusation_id) - (нет неключевых атрибутов)

Минимальное покрытие:

(factor_id, accusation_id) - (нет неключевых атрибутов)

Частичных зависимостей нет, ключ составной.

Транзитивных зависимостей нет, дополнительных атрибутов нет.

Все зависимости определяются ключом.

Нарушений нормальных форм нет.

factor_punishment

Поскольку связь отражает отношение многие-ко-многим и не содержит дополнительных атрибутов, используется составной ключ.

Кандидатный ключ: (factor_id, punishment_id)

Функциональные зависимости:

(factor_id, punishment_id) - (нет неключевых атрибутов)

Частичных зависимостей нет, ключ составной.

Транзитивных зависимостей нет, дополнительных атрибутов нет.

Все зависит от ключа.

Нарушений нормальных форм нет.

court_judge

Поскольку связь отражает принадлежность судьи к суду и не содержит дополнительных атрибутов, используется составной ключ.

Кандидатный ключ: (court_id, human_id)

Функциональные зависимости:

(court_id, human_id) - (нет неключевых атрибутов)

Частичных зависимостей нет, ключ составной.

Транзитивных зависимостей нет, атрибутов кроме ключа нет.

Все атрибуты зависят от ключа.

Нарушений нормальных форм нет.

prison_human

Поскольку один человек может находиться в одной тюрьме несколько раз в разное время, но только в 1 тюрьме в конкретный момент времени, в ключ включается время начала заключения. Дата освобождения определяется этим фактом заключения.

Кандидатный ключ: (human_id, prison_id, imprisoned_time)

Функциональные зависимости:

(human_id, prison_id, imprisoned_time) - released_time

Минимальное покрытие:

(human_id, prison_id, imprisoned_time) - released_time

Частичных зависимостей нет, ключ составной (3 атрибута используются полностью).

Транзитивных зависимостей нет, released_time зависит только от полного ключа.

Все атрибуты зависят от полного ключа.

Нарушений нормальных форм нет.

4. Приведение к 3NF

В исходной схеме есть 2 нарушения 3NF - в court и accusation

1. court

Проблема в транзитивной зависимости, location_name по факту зависит от location_id, а в нынешней схеме - от id и location_id. Из-за чего возникает дублирование.

Функциональные зависимости:

id - name, location_id

location_id - location_name

Поскольку отношение location уже существует в схеме, атрибут location_name должен храниться только в нём.

Для приведения к 3NF из court удаляется избыточный атрибут location_name, и отношение принимает вид:

court(id, name, location_id)

Таким образом устраняется дублирование данных и транзитивная зависимость. А также, аномалии вставки и изменения

```
INSERT INTO court VALUES (1, 'Royal Court', 10, 'London');
```

```
UPDATE court
```

```
SET location_name = 'Greater London'
```

```
WHERE location_id = 10;
```

Теперь location_name различен для 1 конкретного отношения

2. accusation

В отношении accusation нарушение 3NF возникает из-за транзитивной зависимости court_name от court_id.

Функциональные зависимости:

id - human_id, type, accused_time, court_id

court_id - court_name

В результате нормализации court_name удаляется из accusation, его хранение осуществляется в отношении court, где оно зависит от первичного ключа.

Итоговая структура:

accusation(id, human_id, type, accused_time, court_id)

court(id, name, location_id)

Таким образом устраняется дублирование данных и транзитивная зависимость. А также, аномалии вставки и изменения

```
INSERT INTO accusation VALUES (1, 5, 'treason', '1685-06-20', 10, 'Royal Court');
```

```
UPDATE accusation
```

```
SET court_name = 'High Royal Court'
```

```
WHERE court_id = 10;
```

Теперь court_name различен для 1 конкретного отношения

5. Описание изменений после приведения к 3NF

1. Локализация

Зависимость $id \rightarrow location_id \rightarrow location_name$ была разложена:

$id \rightarrow location_id$ осталось в court

$location_id \rightarrow location_name$ осталось только в location

Зависимость $court_id \rightarrow court_name$ из accusation была вынесена в отношение court:

$court_id \rightarrow court_name$ хранится только в court

2. Зависимости нарушающие NF

После декомпозиции исчезли транзитивные зависимости:

- в court больше нет зависимости $id \rightarrow location_name$ через $location_id$
- в accusation больше нет зависимости $id \rightarrow court_name$ через $court_id$

Теперь все неключевые атрибуты зависят только от ключей своих отношений.

3. Изменения набора зависимостей

В исходной схеме были цепочки зависимостей:

$id \rightarrow location_id \rightarrow location_name$

$id \rightarrow court_id \rightarrow court_name$

После приведения к 3NF эти цепочки разорваны, и зависимости стали локальными внутри отдельных отношений:

- court: $id \rightarrow name, location_id$
- location: $location_id \rightarrow name, climate$
- accusation: $id \rightarrow human_id, type, accused_time, court_id$
- court: $court_id \rightarrow name$

6. Проверка на BCNF

Проверка BCNF выполняется по правилу: для каждой нетривиальной функциональной зависимости $X \rightarrow Y$ детерминант X должен быть суперключом отношения.

- **human**

Функциональная зависимость: $id \rightarrow first, middle, last, job, is_sane, courage$

id является ключом (суперключом).

Следовательно, BCNF выполняется.

- **prison**

Функциональная зависимость: $id \rightarrow conditions, location_id$

id является ключом.

Следовательно, BCNF выполняется.

- **location**

Функциональная зависимость: $id \rightarrow name, climate$

id является ключом.

Следовательно, BCNF выполняется.

- **event**

Функциональная зависимость: $id \rightarrow is_successfull, location_id$

id является ключом.

Следовательно, BCNF выполняется.

- **factor**

Функциональная зависимость: $id \rightarrow type, start_time, end_time$

id является ключом.

Следовательно, BCNF выполняется.

- **punishment**

Функциональная зависимость: $id \rightarrow accusation_id, type, execution_time$

id является ключом.

Следовательно, BCNF выполняется.

- **wound**

Функциональная зависимость: $id \rightarrow human_id, is_fatal, dt, is_cured$

id является ключом.

Следовательно, BCNF выполняется.

- **event_people**

Функциональная зависимость: $(event_id, human_id) \rightarrow role, consequence$

(event_id, human_id) является ключом (суперключом).

Следовательно, BCNF выполняется.

- **relationships**

Функциональная зависимость: $(human1_id, human2_id) \rightarrow type$

(пара $human1_id, human2_id$) является ключом.

Следовательно, BCNF выполняется.

- **factor_accusation**

Нет нетривиальных функциональных зависимостей кроме ключа ($factor_id, accusation_id$).

Ключ является суперключом.

Следовательно, BCNF выполняется.

- **factor_punishment**

Аналогично: нет нетривиальных зависимостей кроме ключа ($factor_id, punishment_id$).

Следовательно, BCNF выполняется.

- **court_judge**

Функциональная зависимость: $(court_id, human_id) \rightarrow$ (нет неключевых атрибутов)

Ключ является суперключом.

Следовательно, BCNF выполняется.

- **prison_human**

Функциональная зависимость: (human_id, prison_id, imprisoned_time) → released_time

Составной ключ является суперключом.

Следовательно, BCNF выполняется.

Итог

Во всех отношениях любая функциональная зависимость имеет в качестве детерминанта ключ или суперключ.

Следовательно, вся схема соответствует нормальной форме BCNF.

7. Сравнение схем, денормализация

Сравнение

Исходная схема

В исходной схеме присутствовали:

- дублирование данных (court_name в accusation, location_name в court)
- транзитивные зависимости ($id \rightarrow court_id \rightarrow court_name$, $id \rightarrow location_id \rightarrow location_name$)
- частичная избыточность в некоторых связях

Из-за этого возникали аномалии:

- обновления (нужно менять одно и то же значение в нескольких местах)
- вставки (нельзя добавить запись без связанных данных)
- удаления (потеря информации при удалении строк)

После разбиения на 3NF:

- устранены транзитивные зависимости
- вынесены справочные сущности (location, court)
- убрано дублирование атрибутов

Изменения:

- court стал ссылаться на location через location_id
- accusation перестал хранить court_name
- данные стали храниться в одной “точке истины”

Результат:

- уменьшены аномалии обновления
- структура стала более нормализованной
- улучшена согласованность данных

Денормализация

1. accusation + court

Можно добавить поля court в accusation, так как у любого обвинения будет слушание в суде.

accusation (

id,

human_id,

type,

accused_time,

court_id,

court_name

)

Плюсы:

- ускорение SELECT
- отсутствие необходимости JOIN
- проще запросы на уровне приложения

Минусы:

- появляется избыточность данных

- возникает аномалия обновления (при изменении названия суда нужно обновлять все accusation)
- риск несогласованности данных между court и accusation

8. Триггер

При удалении суда:

- создается событие - court_removed, с участниками - судьями
- судьи, которые работали в этом суде перемещаются в другие суды.
приоритет - та же локация, иначе - случайная
- если у судей, которых расформировали есть судимость - они отправляются в тюрьму

```
CREATE OR REPLACE FUNCTION delete_court()
RETURNS TRIGGER AS $$
DECLARE
    v_event_id INTEGER;
    v_judge RECORD;
    v_new_court_id INTEGER;
    v_has_criminal_record BOOLEAN;
    v_prison_id INTEGER;
    v_max_event_id INTEGER;
BEGIN
    SELECT COALESCE(MAX(id), 0) + 1 INTO v_max_event_id FROM event;

    INSERT INTO event (id, is_successfull, location_id)
    VALUES (v_max_event_id, TRUE, OLD.location_id)
    RETURNING id INTO v_event_id;

    FOR v_judge IN (
        SELECT h.id as human_id, h.first, h.middle, h.last
        FROM court_judge cj
```

```

        JOIN human h ON cj.human_id = h.id

        WHERE cj.court_id = OLD.id

    ) LOOP

        INSERT INTO event_people (event_id, human_id, role, consequence)

        VALUES (v_event_id, v_judge.human_id, null, null);

        SELECT c.id INTO v_new_court_id

        FROM court c

        LEFT JOIN court_judge cj ON c.id = cj.court_id

        WHERE c.location_id = OLD.location_id

            AND c.id ≠ OLD.id

        GROUP BY c.id

        ORDER BY COUNT(cj.human_id) ASC

        LIMIT 1;

        IF v_new_court_id IS NULL THEN

            SELECT c.id INTO v_new_court_id

            FROM court c

            LEFT JOIN court_judge cj ON c.id = cj.court_id

            WHERE c.id ≠ OLD.id

            GROUP BY c.id

            ORDER BY RANDOM()

            LIMIT 1;

        END IF;

        IF v_new_court_id IS NOT NULL THEN

```

```

        INSERT INTO court_judge (court_id, human_id)

        VALUES (v_new_court_id, v_judge.human_id);

    END IF;

    SELECT EXISTS(

        SELECT 1

        FROM accusation a

        WHERE a.human_id = v_judge.human_id

    ) INTO v_has_criminal_record;

    IF v_has_criminal_record THEN

        SELECT p.id INTO v_prison_id

        FROM prison p

        WHERE p.location_id = OLD.location_id

        LIMIT 1;

        IF v_prison_id IS NULL THEN

            SELECT p.id INTO v_prison_id

            FROM prison p

            LIMIT 1;

        END IF;

        IF v_prison_id IS NOT NULL THEN

            INSERT INTO prison_human (human_id, prison_id,
imprisoned_time)

            VALUES (v_judge.human_id, v_prison_id, NOW());

        END IF;

```



```

        END IF;

    END LOOP;

    DELETE FROM court_judge WHERE court_id = OLD.id;

    DELETE FROM punishment
    WHERE accusation_id IN (
        SELECT id FROM accusation WHERE court_id = OLD.id
    );

    DELETE FROM factor_accusation
    WHERE accusation_id IN (
        SELECT id FROM accusation WHERE court_id = OLD.id
    );

    DELETE FROM factor_punishment
    WHERE punishment_id IN (
        SELECT id FROM punishment WHERE accusation_id IN (
            SELECT id FROM accusation WHERE court_id = OLD.id
        )
    );

    UPDATE accusation
    SET court_id = NULL
    WHERE court_id = OLD.id;

```

```

        RETURN OLD;

EXCEPTION

    WHEN OTHERS THEN

        RAISE EXCEPTION 'Error when court removed: %', SQLERRM;

END;

$$ LANGUAGE plpgsql;

CREATE OR REPLACE TRIGGER before_delete_court

    BEFORE DELETE ON court

    FOR EACH ROW

    EXECUTE FUNCTION delete_court();

```

Для работы триггера пришлось изменить создание 2 таблиц, чтобы избежать ошибок FK

```

CREATE TABLE accusation (

    id INT PRIMARY KEY,

    human_id INT,

    type accusation_type,

    accused_time DATE,

    court_id INT,

    FOREIGN KEY (human_id) REFERENCES human(id),

    FOREIGN KEY (court_id) REFERENCES court(id) ON DELETE SET NULL

);

```

```
CREATE TABLE court_judge (  
    court_id INT,  
    human_id INT,  
    PRIMARY KEY (court_id, human_id),  
    FOREIGN KEY (court_id) REFERENCES court(id) ON DELETE CASCADE,  
    FOREIGN KEY (human_id) REFERENCES human(id)  
);
```

Решить проблему в самом триггере невозможно, так как postgres проверяет на ошибки внешних ключей до выполнения триггера

Пример 3NF, но не BCNF

bike_rental

ride_id — id поездки

bike_id — id велосипеда

bike_type — тип велосипеда

Функциональные зависимости

$(ride_id, bike_type) \rightarrow bike_id$

$bike_id \rightarrow bike_type$

Кандидатный ключ: $(ride_id, bike_id)$, $(ride_id, bike_type)$

1NF: нет массивов, пересечений на строках - условие атомарности выполняется

2NF: все 3 атрибута - ключевые, а значит условие запрета частичной зависимости неключевых атрибутов выполняется

3NF:

$(ride_id, bike_type) \rightarrow bike_id$, левая часть - суперключ

$bike_id \rightarrow bike_type$, bike_type - ключевой атрибут

BCNF:

Для любого $X \rightarrow Y$, X суперключ

Википедия: Суперключ — в реляционной модели данных — подмножество атрибутов отношения, удовлетворяющее требованию уникальности: не

существует двух кортежей данного отношения, в которых значения этого подмножества атрибутов совпадают (равны).

Проверим `bike_id` -> `bike_type` на суперключ

`bike_id` - не суперключ, так как он не определяет `ride_id`. Один и тот же велосипед может быть использован в разных поездках

Заключение

Во время выполнения лабораторной работы я познакомился нормализацией и денормализацией, научился писать триггеры.